Beating hash tables with trees? The ART-ful radix trie

November 3, 2018

The Adaptive Radix Tree: ARTful Indexing for Main-Memory Databases

Leis et. al., ICDE 2013 [paper (https://db.in.tum.de/~leis/papers/ART.pdf)]

Tries are an unloved third data structure for building key-value stores and indexes, after search trees (like B-trees (https://en.wikipedia.org/wiki/B-tree) and red-black trees (https://en.wikipedia.org/wiki/Red%E2%80%93black_tree)) and hash tables. Yet they have a number of very appealing properties that make them worthy of consideration - for example, the height of a trie is independent of the number of keys it contains, and a trie requires no rebalancing when updated. Weighing against those advantages is the heavy memory cost that vanilla radix tries can incur, because each node contains a pointer for every possible value of the 'next' character in the key. With ASCII as an example, that's 256 pointers for every node in the tree.

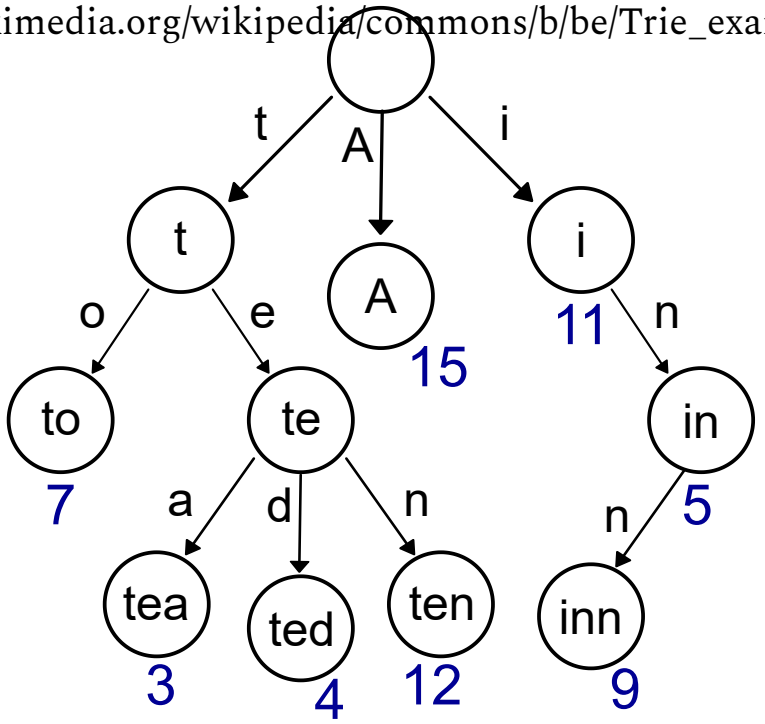
But the astute reader will feel in their bones that this is naive - there must be more efficient ways to store a set of pointers, indexed by a fixed size set of keys (the trie's alphabet). Indeed, there are - several of them, in fact, distinguished by the number of children the node *actually* has, not just how many it might *potentially* have.

This is where the *Adaptive Radix Tree* (ART) comes in. In this breezy, easy-to-read paper, the authors show how to reduce the memory cost of a regular radix trie by *adapting* the data structure used for each node to the number of children that it needs to store. In doing so they show, perhaps surprisingly, that the amount of space consumed by a single key can be bounded no matter how long the key is.

Tries

Very quickly, let's describe tries. Instead of using the entire value of a key to find a place in a tree structure (like, for example, binary search trees, where you compare the whole key to the current node's value), a trie breaks down a key into a sequence of *characters*, and makes one node per character. Each node has a possible child for every character in the alphabet.

(https://upload.wikimedia.org/wikipedia/commons/b/be/Trie_example.svg)



A simple trie over English words (A, to, tea, ted, ten, inn), from Wikipedia.

You find a key in a trie by looking for the first character in the root node - that will point you to a child node. You then look in *that* node for the second character, which will give you another child node, in which you look for the *third* character, and so on. If you get to the end of your key, and you've found every character, you know the key is in the set (NB: this is only true for fixed-length keys, see below for more discussion on key lengths).

Tries have some lovely properties, and the paper does a great job of listing them, and comparing tries against search trees. For example, the number of nodes you have to visit to search for your key depends *on the length of the key* and not on the number of nodes in the trie! If a character is s bits, and your key is k bits long, it will take at most k/s nodes to determine if the key is in the trie.

The paper shows that there is a crossover point where that number is guaranteed to be smaller than an equivalent, perfectly balanced, binary search tree. Their

argument is a bit harder to follow than this simple one: a perfectly balanced binary search tree has depth $log_2 N$ (N is the number of keys in the tree or trie). So we need to know the value of N for which $log_2 N > k/s$, which is true when $N > 2^{k/s}$.

In concrete terms, let's say we have 64-bit keys, and are using 8-bits per character (standard ASCII). Then a trie will be shallower than a binary search tree when they contain more than 256 entries. Pretty compelling!

What's wrong with ordinary tries?

Let's imagine how we might implement a trie, as simply as we can. A single node in a trie corresponds to a *key prefix*; a sequence of characters that is a prefix of one or more keys in the tree. But we know what its prefix is by the path we took to get to it, so we don't need to store it. All we need is a way to find its children.

```
struct TrieNode {
    TrieNode* children[256];
}
```

Not much to it! We need to know how to get to all the nodes that represent key prefixes one character longer than the current one, and that's what the `children` array is for. Here we're assuming an 8-bit alphabet, with 256 possible characters per letter, or a *fan-out* of 256.

The paper calls the width of a single character the *span* of the trie, and it's a critical parameter as it determines a trade-off between the fan-out of each node (and therefore the size of the array in `TrieNode` above), and the height of the trie, since if you can pack more of a value into a single node by having a larger span, you need fewer nodes to describe the whole string. We'll talk more about the span below, but for now you can assume that tries perform best when the span is more than just a couple of bits.

The problem, as you can no-doubt see, is that there's a possibility of a lot of wasted space in each node. Imagine a radix trie containing `foo`, `fox` and `fat`. The root node would have one valid pointer, to `f`, which would have two valid pointers, to `a` and `o`. `a` would have a pointer to `t`, and `o` would have pointers to `o` and `x`.

So our trie would have 6 nodes, but a total of 6 * 26 = 156 pointers, of which 150 / 156 = 96% are empty and therefore wasted space! At 8-bytes per pointer, that's already over 1K wasted space to store just 9 bytes of information.

already over 1K wasted space to store just 9 bytes of information.

This example may seem to be contrived, partly because it is. But the memory overhead of tries is real, and the paper establishes this in Fig 3, which plots the height of the trie against the space required to store it for a keyset of 1M 32-bit values. The span is varied, which changes both the height and the storage cost. At a span of 8-bits, the trie has height 4, but takes more than 128MB to store. As the span gets larger, the memory overhead gets much, much worse for little gain in the height of the trie; if the span is reduced the tree height gets larger very quickly for not much memory benefit.

That wasted space is not just bad use of memory - which is in and of itself less and less concerning nowadays - but has real performance consequences because it reduces the number of nodes that can be kept in the L1 cache. Bringing it under control is the key to ART, and unlocks its biggest performance gains.

Adaptive node structures

The most significant change that ART makes to the standard trie structure is that it introduces the ability to change the datastructure used for each internal node depending on how many children the node actually has, rather than how many it might have. As the number of children in a node changes, ART swaps out one implementation of the node structure for another.

The paper distinguishes four different cases, for nodes with up to 4, 16, 48 and 256 children respectively. Assume we have a union type like the following:

```
union Node {
    Node4* n4;
    Node16* n16;
    Node48* n48;
    Node256* n256;
}
```

This allows us to talk about all the different types of `Node`, and pointers to them, in the following without trying to make them all inherit from a base class. (This elides a few details, check this C implementation of ART (https://github.com/armon/libart) for a fully realised version).

`Node4`

For nodes with up to four children, ART stores all the keys in a list, and the child pointers in a parallel list. Looking up the next character in a string means searching the list of child keys, and then using the index to look up the corresponding pointer. Although this is superficially a less efficient lookup

algorithm, the small size of the child keys arrays means that the constant factor will dominate.

```
struct Node4 {
    char child_keys[4];
    Node* child_pointers[4];
}

Node* find_child(char c, Node4* node) {
    Node* ret = NULL;
    for (int i = 0; i < 4; ++i) {
        if (child_keys[i] == c) ret = node->child_pointers[i];
    }

    return ret;
}
```

Pointers are assumed to be 8 bytes, so a single `Node4` is 36 bytes, so sits in a single cache line. The search loop can also be unrolled. Finally, by not early-exiting from the loop, we can hint to the compiler that it need not use a full branch, but can just use a conditional `cmov` predicated instruction (https://en.wikipedia.org/wiki/Predication_(computer_architecture)). (These optimisations are possible here if we can initialise `child_keys` to some distinguished value that can't be found in any key). The locality benefits make this a very efficient structure for very sparsely populated trie nodes.

`Node16`

Nodes with from 5 to 16 children have an identical layout to `Node4`, just with 16 children per node:

```
struct Node16 {
    char child_keys[16];
    Node* child_pointers[16];
    byte num_children;
}
```

Keys in a `Node16` are stored sorted, so binary search could be used to find a particular key. Since there are only 16 of them, it's also possible to search all the keys in parallel using SIMD. What follows is an annotated version of the algorithm presented in the paper's Fig 8.

algorithm presented in the paper's Fig 8.

```
    // Find the child in `node` that matches `c` by examining all child nodes, in par
allel.
    Node* find_child(char c, Node16* node) {
        // key_vec is 16 repeated copies of the searched-for byte, one for every poss
ible position
        // in child_keys that needs to be searched.
        __mm128i key_vec = _mm_set1_epi8(c);

        // Compare all child_keys to 'c' in parallel. Don't worry if some of the keys
aren't valid,
        // we'll mask the results to only consider the valid ones below.
        __mm128i results = _mm_cmpeq_epi8(key_vec, node->child_keys);

        // Build a mask to select only the first node->num_children values from the c
omparison
        // (because the other values are meaningless)
        int mask = (1 << node->num_children) - 1;

        // Change the results of the comparison into a bitfield, masking off any inva
lid comparisons.
        int bitfield = _mm_movemask_epi8(results) & mask;

        // No match if there are no '1's in the bitfield.
        if (bitfield == 0) return NULL;

        // Find the index of the first '1' in the bitfield by counting the leading ze
ros.
        int idx = ctz(bitfield);

        return node->child_pointers[idx];
    }
```

This is superior to binary-search: no branches (except for the test when bitfield is 0), and all the comparisons are done in parallel. The `Node16` takes up 145 bytes, but the `child_keys` field is only 16 bytes.

Node48

The next node can hold up to three times as many keys as a `Node16`. As the paper says, when there are more than 16 children, searching for the key can become expensive, so instead the keys are stored implicitly in an array of 256 indexes. The

entries in that array index a separate array of up to 48 pointers.

```
struct Node48 {
    // Indexed by the key value, i.e. the child pointer for 'f'
    // is at child_ptrs[child_ptr_indexes['f']]
    char child_ptr_indexes[256];

    Node* child_ptrs[48];
    char num_children;
}
```

The idea here is that this is superior to just storing an array of 256 `Node` pointers because you can store 48 children in 640 bytes (where 256 pointers would take 2k). Looking up the pointer does take an extra indirection:

```
Node* find_child(char c, Node48* node) {
    int idx = node->child_ptr_indexes[c];
    if (idx == -1) return NULL;

    return node->child_ptrs[idx];
}
```

The paper notes that in fact only 6 bytes (i.e. $log_2(48)$) are needed for each index; both in the paper and here it's simpler to use a byte per index to avoid any shifting and masking.

`Node256`

The final node type is the traditional trie node, used when a node has between 49 and 256 children.

```
struct Node256 {
    Node* child_ptrs[256];
}

Node* find_child(char c, Node256* node) {
    return child_ptrs[c];
}
```

Looking up child pointers is obviously very efficient - the most efficient of all the node types - and when occupancy is at least 49 children the wasted space is less significant (although not 0 by any stretch of the imagination).

Storing values

"But wait!" you might reasonably say. "I see how we can store keys, but where are the *values*?"

The easiest thing to do is to augment every `Node` structure with its own `Value*`. If that pointer is not null, then you know the key which ends at the current `Node` a) exists and b) has the pointed-to value.

The paper eschews this approach, and I can only assume it's because the extra 8-bytes per `Node` is a large price to pay when that pointer is - most of the time - probably going to be `NULL`.

Instead, all values are stored in one of three ways:

- **Single value** nodes are leaf nodes which store… exactly one value. They'd be represented in our scheme as a different `Node` type.
- **Multi-value** leaves are just like the regular `Node[4|16|48|256]` types, but the `child_ptrs` now become an array of `Value*`.
- **Pointer/value slots** store values *directly* in the slots otherwise used for child pointers, and distinguishes between them based on the highest bit - let's say 0 to interpret the pointer as a child node pointer, and 1 to interpret it as a pointer to a value. Using these high bits doesn't lose us anything on a modern CPU whose addressable memory is 'only' 2^{48} bytes - the extra 16 bits in a 64-bit pointer can be used to store extra information.

At this point you might share the same confusion that I did: all these value-storage designs *replace* a child node (or a pointer to a child node) with a leaf node (or a pointer to a value). That is, once you get to the end of a key that's in the trie, you can't go any further - but what about keys that are prefixes of some other key? What about `http://google.com` and `http://google.com/chrome`?

Our original idea of storing `Value` pointers in every node addresses this problem by keeping node and value pointers separate. But if we're not going to do that, are we just confined to keysets where no key is the prefix of any other?

The answer is yes - but it's not much of a restriction. There are basically two cases:

- **Fixed-length datatypes** such as 128-bit integers, or strings of exactly 64-bytes, don't have any problem because there can, by construction, never be any key that's a prefix of any other.

- **Variable-length datatypes** such as general strings, can be transformed into types where no key is the prefix of any other by a simple trick: *append the NULL byte to every key*. The NULL byte, as it does in C-style strings, indicates that this is the end of the key, and no characters can come after it. Therefore no string with a null-byte can be a prefix of any other, because no string can have any characters after the NULL byte!

This fact is slightly hidden in the paper in the discussion (in section IV.B) about transforming data types to an equivalent one that can be lexicographically ordered (this property is required for the trie to support range queries).

> *"... it is important that each string is terminated with a value which does not appear anywhere else in any string (e.g. the 0 byte). The reason is that* ***keys must not be prefixes of any other keys****."*

Other tricks: lazy compression and path expansion

Two techniques are described that allows the trie to shrink its height if there are nodes that only have one child. The paper claims these are "well-known", and therefore not novel, but they reduce the impact of having either strings with a unique suffix (lazy expansion) or a common prefix (path compression). Figure 6 in the paper illustrates both techniques clearly.

The way I read the paper, both techniques address different aspects of the problem: when you have a sequence of nodes with only one child (so that sequence of nodes encodes only one string of characters), why not collapse that sequence into one node, and avoid the overhead of storing a node per character?

The paper addresses two different instances of the same problem.

When the sequence of nodes ends in a leaf, *lazy expansion* is used - don't bother to write out the full sequence, just create a single node that has the key suffix and the value. Implementing lazy expansion really just means having a separate `Node` type that has a key pointer and a value pointer. If this node type is encountered during a query, you can just compare the key to the searched-for key character-by-character. If this node type is found during an insertion, it has to be swapped out for a 'real' node, and each suffix of the existing key and the new one needs to be inserted (both will probably wind up with a lazily-expanded node representing

them; it's instructive to think of why that is).

If, instead, the sequence of single-child nodes does *not* end in a leaf, ART uses *path compression*. This also collapses the sequence of nodes, but into the node at the end of the sequence that has more than one child. That is, consider inserting `CAST` and `CASH` into an empty trie; path compression would create a node at the root that has the `CAS` prefix, and two children, one for `T` and one for `H`. That way the trie doesn't need individual nodes for the `C->A->S` sequence.

Evaluation notes and wrap-up

Usually when one considers any tree-based search structure, they do so only if range queries are needed, because otherwise the received wisdom is that a hash table's performance will wipe the floor with the tree. What is particularly interesting about ART, then, is that for some workloads it is *competitive with, or even* ***beats*** *chained hash tables for point-lookup queries*.

The results are laid out in Figure 10, but the gist is that, for random lookups, ART performs better than a chained hash table implementation when the key set is 'dense'; i.e. all integers from $1..N$ are in the set. When the key set is sparse - i.e. it includes randomly selected integers from a much bigger range - performance drops (but is still better than all other data structures included in the comparison, particularly as the number of keys gets larger).

Of course, when the key set is dense, most nodes will be `Node256` instances, and so ART isn't too different from an ordinary trie.

Things get more interesting when the access pattern is skewed, as it allows ART to take advantage of cache effects, whereas hash tables have no data locality when there is locality in the query set. As the skew gets significant (see Figure 12), ART performs nearly 50% faster than a hash table!

Given these results, it was a bit disappointing not to see similar experiments for range queries. Indeed, range queries are treated as a bit of an afterthought throughout the paper - there's no pseudocode for them and they appear to only be measured indirectly as part of the TPC-C workload. In that case, ART is shown to dominate both a binary search tree (a Red-Black tree, here) or a hybrid hash table / RB-tree combination (hash tables alone don't support efficient range queries).

Finally, there's a brief consideration of space: ART uses less than half the space

required for the hybrid index for the TPC-C workload.

The paper doesn't include a discussion of thread-safety (but measures concurrent read performance!). That's a topic for a future (https://db.in.tum.de/~leis/papers/artsync.pdf) paper.

ART has been evaluated many times since the paper was published (e.g. this recent takedown of the Bw-tree), and is still considered an *extremely* competitive index implementation. It's appealing not only for its performance, but for the

minimal delta from a well-understood data structure that it represents. You could implement a basic version of ART in a few hours of work. So why not… trie?

← PREVIOUS POST (HTTPS://WWW.THE-PAPER-TRAIL.ORG/POST/MICA-PAPER-NOTES/)

NEXT POST → (HTTPS://WWW.THE-PAPER-TRAIL.ORG/POST/2020-04-06-PHYSALIA/)

(mailto:henry.robinson+papertrail@gmail.com) (https://twitter.com/henryr)

Henry Robinson · © 2020 · The Paper Trail: Distributed systems and database research (https://www.the-paper-trail.org)

Hugo v0.72.0 (http://gohugo.io) powered • Theme by Beautiful Jekyll (http://deanattali.com/beautiful-jekyll/) adapted to Beautiful Hugo (https://github.com/halogenica/beautifulhugo)